

UNIVERSITAS SEBELAS MARET - SEKOLAH VOKASI
PROGRAM STUDI D3 TEKNIK INFORMATIKA K KAB. MADIUN
PRAKTIK PEMROGRAMAN FRONT END ENGINEERING

MODUL PRAKTIKUM: MANAJEMEN STATE MODERN (CLIENT UI STATE VS SERVER STATE)

BAB 7: MANAJEMEN STATE MODERN — SEPARASI CLIENT UI STATE (ZUSTAND) & SERVER STATE (TANSTACK QUERY)

A. Deskripsi Singkat Mata Kuliah & Praktikum

Modul praktikum ini membahas arsitektur manajemen keadaan (State Management) modern dalam pengembangan aplikasi web berkinerja tinggi. Dalam beberapa dekade terakhir, industri perangkat lunak beroperasi menggunakan paradigma Global State Store monolitik seperti Redux klasik, di mana seluruh data aplikasi baik status modal antarmuka maupun hasil pemanggilan REST API eksternal disimpan dalam satu pohon state raksasa. Namun, evolusi lanskap Front-End menunjukkan bahwa penggabungan ini memicu hambatan kompleksitas (*boilerplate tax*), ketidakstabilan re-render, serta tingginya beban sinkronisasi status server.

Praktikum ini melatih mahasiswa D3 Teknik Informatika Sekolah Vokasi UNS untuk mengimplementasikan pemisahan tegas antara Client UI State dan Server State. Mahasiswa akan menguasai penggunaan Zustand (~0.5 KB gzipped)—pustaka state management mikro yang berbasis selector presisi—untuk mengelola status antarmuka lokal (seperti tema, drawer, dan draf formulir). Secara berdampingan, mahasiswa mengintegrasikan TanStack Query v5 (React Query) untuk menangani Server State meliputi pencarian data, alokasi memori cache, invalidasi query otomatis, dan refetching asinkron. Pendekatan ini menjamin kode bersih, performa re-render optimal, serta meminimalkan potensi bugs pada sistem berskala industri.

B. Tujuan Praktikum & Capaian Pembelajaran (CPMK)

- **Menganalisis Taksonomi State Management:** Mahasiswa mampu membedakan karakteristik, kebutuhan sinkronisasi, dan masa hidup (lifecycle) antara Client UI State dan Server State secara akurat.
- **Penguasaan Zustand untuk Client UI State:** Mahasiswa mampu merancang dan mengimplementasikan store Zustand (~0.5 KB) tanpa Context Provider wrapper dengan selector re-render yang presisi.
- **Penguasaan TanStack Query v5 untuk Server State:** Mahasiswa mampu mengelola siklus hidup data remote API meliputi caching, staleTime, gcTime, invalidasi query, serta pemutakhiran responsif (optimistic updates).
- **Penghilangan Anti-Pattern State Centralization:** Mahasiswa dapat mengeliminasi kesalahan arsitektur berupa penyimpanan data asynchronous API di dalam toko global client biasa.

- **Integrasi Type-Safe & Runtime Validation:** Mahasiswa mampu memadukan Strict TypeScript dan skema validasi Zod untuk menjamin integritas tipe data antara layer TanStack Query dan Zustand.
- **Pengujian Performa & Verifikasi Kode Statis:** Mahasiswa mampu mengaudit siklus re-render komponen dan memverifikasi kualitas kode menggunakan analisis statis SonarQube guna menekan utang teknis.

C. Dasar Teori Komprehensif

A. Evolusi Arsitektur State Management: Mitos Global Store Monolitik

Pada era awal pengembangan aplikasi Single Page Application (SPA) menggunakan React, terdapat anggapan umum bahwa seluruh keadaan aplikasi harus disimpan dalam satu wadah tersentralisasi (Single Source of Truth) seperti Redux klasik. Dalam pola ini, ketika sebuah aplikasi melakukan pemanggilan REST API untuk mengambil daftar tugas akademis, respons JSON disimpan di dalam Redux store bersama-sama dengan status 'isModalOpen' atau 'themeColor'. Pendekatan monolitik ini menimbulkan tantangan struktural yang serius:

- **Boilerplate Tax & Over-Engineering:** Pengembang diwajibkan menuliskan konstanta action types, action creators, reducers, dan mapDispatchToProps hanya untuk mengubah sebuah flag Boolean antarmuka.
- **Manual Cache Management:** Data yang diambil dari server memerlukan logika manual untuk melacak status loading, penanganan error, re-fetching saat koneksi terputus, serta pembersihan cache yang kadaluarsa.
- **Re-render Cascade:** Perubahan kecil pada satu bagian state yang tersentralisasi berisiko memicu kalkulasi ulang dan re-render pada komponen-komponen antarmuka yang tidak relevan jika selector tidak dimemoisasi dengan sempurna.

Seiring berkembangnya skala aplikasi web, komunitas rekayasa perangkat lunak menyadari bahwa data yang dikelola dalam aplikasi web modern sebenarnya terbagi menjadi dua kategori utama dengan sifat arsitektural yang bertolak belakang: Client UI State dan Server State.

Dimensi Arsitektur	Client UI State	Server State
Kepemilikan Data (Ownership)	Sepenuhnya dimiliki & dikontrol oleh browser client secara lokal.	Dimiliki oleh basis data server remote; client hanya menyimpan salinan sementara (cache).
Sifat Eksekusi (Asynchrony)	Sinkron (Synchronous) & instan. Perubahan status langsung tercermin.	Asinkron (Asynchronous). Bergantung pada latensi jaringan & status koneksi.
Masa Hidup (Lifecycle)	Ephemeral / Sementara. Hilang saat tab di-refresh (kecuali dipersistikan).	Persisten di sisi server. Memerlukan strategi stale-time, refetching, & invalidasi.
Masalah Utama (Core Challenge)	Mencegah re-render tak perlu & efisiensi alokasi memori UI.	Menjaga konsistensi data cache client agar tetap sinkron dengan data asli server.

Pustaka Ideal (Tooling)	Zustand (~0.5 KB) atau React Context lokal.	TanStack Query v5 (React Query) atau RTK Query.
--------------------------------	---	---

B. Arsitektur Zustand untuk Client UI State

Zustand adalah pustaka manajemen keadaan antarmuka (Client UI State) yang sangat ringan (~0.5 KB gzipped) dan berbasis prinsip publish-subscribe sederhana. Berbeda dengan Redux Toolkit (RTK) yang berukuran sekitar 17 KB gzipped serta mewajibkan pembungkusan komponen dalam `<Provider store={store}>`, Zustand bekerja secara unopinionated dan dapat dipanggil langsung dari mana saja tanpa Context Provider.

Keunggulan utama Zustand terletak pada presisi re-render berbasis selector. Ketika sebuah komponen berlangganan pada potongan state tertentu (slice), komponen tersebut hanya akan di-render ulang jika nilai spesifik yang dipilih mengalami perubahan (berdasarkan perbandingan kesamaan `Object.is`). Kebutuhan memori yang sangat kecil (~0.5 KB) menjadikan Zustand solusi ideal untuk mengelola variabel antarmuka seperti status drawer, pergantian tema gelap/terang, atau filter tabel interaktif.

C. Arsitektur TanStack Query v5 untuk Server State Management

TanStack Query (React Query) secara mendasar bukan sekadar alat pembantu pemanggilan API, melainkan sebuah layer pengelolaan cache data asynchronous yang canggih di sisi client. TanStack Query mengotomatisasi seluruh kompleksitas pengambilan data dari remote server dengan menyediakan fungsi-fungsi bawaan:

- **Siklus Stale-While-Revalidate:** Data yang diambil dari server disimpan di memori cache dengan atribut `'staleTime'` (durasi data dianggap segar) dan `'gcTime'` (durasi penyimpanan cache tak terpakai sebelum di-garbage collect).
- **Automatic Re-fetching:** Query secara otomatis melakukan penghentian data usang dan melakukan re-fetch tak terlihat saat aplikasi mendapatkan kembali fokus jendela browser (`refetchOnWindowFocus`) atau ketika koneksi internet terhubung kembali (`refetchOnReconnect`).
- **Deduplikasi Permintaan (Query Deduplication):** Jika lima komponen terpisah meminta data dengan query key yang sama (misalnya `['tasks']`) secara bersamaan, TanStack Query hanya mengeksekusi satu permintaan jaringan tunggal dan membagikan hasilnya ke seluruh komponen.
- **Invalidasi Query Presisi (Query Invalidation):** Setelah eksekusi mutasi data (Create, Update, Delete) berhasil, developer cukup memanggil `queryClient.invalidateQueries({ queryKey: ['tasks'] })` untuk memperbarui antarmuka secara konsisten.

D. Tren Adopsi Industri & Statistik 2026

Berdasarkan data tren unduhan NPM pertengahan 2026, terjadi pergeseran masif di mana Zustand meraih sekitar 42 juta unduhan mingguan, melampaui Redux Toolkit yang mencatatkan sekitar 22 juta unduhan mingguan. Penyebab utama dari pergeseran ini adalah

pemisahan peran yang semakin jelas: pengembang web modern menyerahkan seluruh pengelolaan Server State kepada TanStack Query / RTK Query, sementara sisa Client UI State yang berukuran kecil dikelola secara efisien menggunakan Zustand.

E. Integration Type-Safe & Runtime Validation dengan Zod

TypeScript memberikan keamanan tipe data kompilasi, namun data yang diterima dari REST API bersifat tidak pasti (unknown) pada tingkat runtime. Dalam arsitektur TanStack Query modern, penggunaan pustaka validasi skema seperti Zod sangat dianjurkan. Dengan mendefinisikan skema Zod (`z.object({...})`) dan mengekstrak tipenya via `z.infer`, setiap data API yang masuk diproses secara valid sebelum disimpan ke dalam memori cache TanStack Query, menjamin keamanan tipe data dari ujung ke ujung (End-to-End Type Safety).

F. Verifikasi Kode Statis & Penjaminan Kualitas (SonarQube)

Penggunaan kuis dan interaksi state yang kompleks rentan menimbulkan bug tersembunyi seperti **stale closures** atau **infinite re-render loops**. Laporan riset SonarSource (2026) mengungkapkan bahwa 96% pengembang tidak sepenuhnya mempercayai kebenaran fungsional kode tanpa verifikasi, dan 70% tim memanfaatkan perkakas analisis statis (Static Analysis Tools seperti SonarQube). Dalam praktikum ini, aturan linting statis diterapkan untuk memverifikasi kesesuaian Query Keys, penanganan status error async, dan deteksi kebocoran memori pada Zustand store.

D. Langkah-Langkah Praktikum Terpandu (Studi Kasus Proyek)

Pada praktikum ini, mahasiswa akan membangun sistem dashboard akademis interaktif berjudul "UNS Smart Task & Project Management Dashboard". Aplikasi ini secara eksplisit menerapkan pemisahan tugas: Zustand mengelola status UI lokal (penyaringan kategori, status drawer, dan tema antarmuka), sedangkan TanStack Query v5 mengelola pemanggilan API async, caching data tugas, serta pemutakhiran data (mutasi).

A. Kode Berkas: `src/types/task.ts`

```
// src/types/task.ts
import { z } from 'zod';

export const TaskSchema = z.object({
  id: z.string().uuid(),
  title: z.string().min(3, "Judul tugas minimal 3 karakter"),
  category: z.enum(["Praktikum", "Teori", "Proyek Akhir"]),
  status: z.enum(["Pending", "In_Progress", "Completed"]),
  dueDate: z.string()
});

export const CreateTaskSchema = TaskSchema.omit({ id: true });

export type Task = z.infer<typeof TaskSchema>;
export type CreateTaskInput = z.infer<typeof CreateTaskSchema>;
```

B. Kode Berkas: `src/store/useUIStore.ts`

```
// src/store/useUIStore.ts
import { create } from 'zustand';

interface UIState {
  isSidebarOpen: boolean;
  selectedCategory: string; // 'All' | 'Praktikum' | 'Teori' | 'Proyek Akhir'
  themeMode: 'light' | 'dark';
  toggleSidebar: () => void;
  setSelectedCategory: (category: string) => void;
  toggleTheme: () => void;
}

// Store Zustand (~0.5 KB) murni untuk Client UI State
export const useUIStore = create<UIState>()((set) => ({
  isSidebarOpen: true,
  selectedCategory: 'All',
  themeMode: 'light',
  toggleSidebar: () => set((state) => ({ isSidebarOpen: !state.isSidebarOpen })),
  setSelectedCategory: (category) => set({ selectedCategory: category }),
  toggleTheme: () => set((state) => ({ themeMode: state.themeMode === 'light' ? 'dark' :
'light' })))
}));
```

C. Kode Berkas: src/services/taskApi.ts

```
// src/services/taskApi.ts
import { Task, TaskSchema, CreateTaskInput } from '../types/task';
import { z } from 'zod';

const API_URL = 'https://api.uns.ac.id/v1/tasks';

export const fetchTasks = async (): Promise<Task[]> => {
  const response = await fetch(API_URL);
  if (!response.ok) throw new Error("Gagal mengambil data tugas dari server");
  const rawData = await response.json();
  // Validasi skema runtime menggunakan Zod
  return z.array(TaskSchema).parse(rawData);
};

export const createTask = async (input: CreateTaskInput): Promise<Task> => {
  const response = await fetch(API_URL, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(input)
  });
  if (!response.ok) throw new Error("Gagal menambah tugas baru");
  const rawData = await response.json();
  return TaskSchema.parse(rawData);
};
```

D. Kode Berkas: src/hooks/useTasksQuery.ts

```
// src/hooks/useTasksQuery.ts
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import { fetchTasks, createTask } from '../services/taskApi';
import { CreateTaskInput } from '../types/task';

export const TASK_QUERY_KEY = ['tasks'];
```

```
// Custom Hook TanStack Query v5 untuk Server State Fetching
export const useTasksQuery = () => {
  return useQuery({
    queryKey: TASK_QUERY_KEY,
    queryFn: fetchTasks,
    staleTime: 1000 * 60 * 5, // Data dianggap segar selama 5 menit
    gcTime: 1000 * 60 * 15,   // Cache disimpan selama 15 menit
  });
};

// Custom Hook untuk Mutasi Data dengan Invalidasi Cache Otomatis
export const useCreateTaskMutation = () => {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: createTask,
    onSuccess: () => {
      // Menginvalidasi cache 'tasks' agar TanStack Query mere-fetch data terbaru secara
      konsisten
      queryClient.invalidateQueries({ queryKey: TASK_QUERY_KEY });
    }
  });
};
```

E. Kode Berkas: `src/components/TaskDashboard.tsx`

```
// src/components/TaskDashboard.tsx
import React from 'react';
import { useUIStore } from '../store/useUIStore';
import { useTasksQuery, useCreateTaskMutation } from '../hooks/useTasksQuery';

export const TaskDashboard: React.FC = () => {
  // 1. Ambil Client UI State dari Zustand store (Selector Presisi)
  const isSidebarOpen = useUIStore((s) => s.isSidebarOpen);
  const selectedCategory = useUIStore((s) => s.selectedCategory);
  const toggleSidebar = useUIStore((s) => s.toggleSidebar);
  const setSelectedCategory = useUIStore((s) => s.setSelectedCategory);

  // 2. Ambil Server State & Mutasi dari TanStack Query hook
  const { data: tasks, isLoading, isError, error } = useTasksQuery();
  const createTaskMutation = useCreateTaskMutation();

  // Filter tugas berdasarkan Client State category
  const filteredTasks = tasks?.filter((t) =>
    selectedCategory === 'All' ? true : t.category === selectedCategory
  );

  return (
    <div className={`dashboard-layout ${isSidebarOpen ? 'sidebar-expanded' : 'sidebar-collapsed'}`}>
      <header className="flex justify-between items-center p-4 bg-slate-800 text-white">
        <button onClick={toggleSidebar} className="btn-toggle">
          {isSidebarOpen ? 'Sembunyikan Sidebar' : 'Tampilkan Sidebar'}
        </button>
        <h1 className="text-xl font-bold">UNS Task Dashboard</h1>
      </header>

      <div className="flex">
        {isSidebarOpen && (
```

```

<aside className="w-64 p-4 bg-slate-100 border-r">
  <h3 className="font-semibold mb-2">Filter Kategori (Client UI State)</h3>
  {[ 'All', 'Praktikum', 'Teori', 'Proyek Akhir' ].map((cat) => (
    <button
      key={cat}
      onClick={() => setSelectedCategory(cat)}
      className={`block w-full text-left p-2 rounded mb-1 ${
        selectedCategory === cat ? 'bg-blue-600 text-white' : 'bg-white'
      }`}
    >
      {cat}
    </button>
  ))}
</aside>
)}

<main className="flex-1 p-6">
  <h2 className="text-lg font-bold mb-4">Daftar Tugas Akademis (Server State)</h2>

  { /* Status Penanganan Server State */ }
  {isLoading && <div className="p-4 bg-yellow-100">Memuat data dari server...</div>}
  {isError && <div className="p-4 bg-red-100 text-red-700">Error: {(error as
Error).message}</div>}

  {filteredTasks && (
    <ul className="space-y-2">
      {filteredTasks.map((task) => (
        <li key={task.id} className="p-3 border rounded bg-white flex justify-between">
          <span><strong>[{task.category}]</strong> {task.title}</span>
          <span className="badge">{task.status}</span>
        </li>
      ))}
    </ul>
  )}
</main>
</div>
</div>
);
};

```

E. Tugas Praktikum Mandiri (Case Project SRS Mahasiswa)

Dengan mengasumsikan bahwa setiap mahasiswa telah memiliki topik proyek aplikasi mandiri beserta dokumen Software Requirements Specification (SRS), setiap mahasiswa diwajibkan mengimplementasikan arsitektur pemisahan Client UI State dan Server State secara penuh pada proyek aplikasinya.

A. Poin Persyaratan Komponen Wajib Proyek Tim

- **Penerapan Zustand Store (~0.5 KB) untuk Client UI State:** Membuat minimal 1 store Zustand yang mengelola minimal 3 atribut status antarmuka lokal (seperti Active Modal/Drawer, Theme Toggle, Active Navigation Tab, atau Draft Form Steps). Tidak diperbolehkan menggunakan React Context global untuk variabel UI.

- **Penerapan TanStack Query v5 untuk Server State:** Mengimplementasikan pemanggilan REST API menggunakan custom hooks `useQuery` untuk minimal 2 entitas data utama proyek (misal: Produk, Pesanan, User, atau Materi).
- **Konfigurasi Caching & Stale Time:** Menentukan nilai `staleTime` dan `gcTime` yang rasional pada tiap query sesuai karakteristik frekuensi perubahan data pada SRS proyek.
- **Penerapan Mutasi Data & Invalidasi Cache (useMutation):** Membuat minimal 1 fungsi mutasi (Create/Update/Delete) yang secara otomatis melakukan invalidasi query (`invalidateQueries`) untuk memperbarui tampilan antarmuka secara konsisten.
- **Type Safety dengan Zod Schema Validation:** Seluruh data masukan form dan respons REST API wajib diverifikasi menggunakan skema Zod dengan derivasi tipe otomatis (`z.infer`).
- **Penanganan Status Async UI (Loading, Error, Empty):** Antarmuka antarmuka harus secara transparan menyajikan komponen indikator Loading (Skeleton UI), Error Banner, dan Empty State saat Server State diproses.
- **Matriks Analisis Pemisahan State (PDF 1–2 Halaman):** Menyusun tabel matriks yang memetakan seluruh variabel state pada proyek ke dalam kategori Client UI State (Zustand) vs. Server State (TanStack Query) beserta alasannya.

5.2 Rubrik Penilaian Tugas Praktikum

Kriteria Penilaian	Bobot	Indikator Keberhasilan & Kualitas
Arsitektur Zustand (Client State)	25%	Store Zustand bersih, selector presisi tanpa re-render berlebih, tidak mencampurkan data API server ke Zustand.
TanStack Query & Cache Invalidation	30%	Aturan queryKey terstruktur, <code>staleTime</code> tepat, mutasi berhasil menginvalidasi cache otomatis tanpa reload browser.
Type Safety & Zod Runtime Validation	20%	Skema Zod tervalidasi pada API fetching dan form input. Tidak ada penggunaan tipe <code>any</code> .
Penanganan Async UI & UX	15%	Tampilan antarmuka responsif menyajikan Loading Skeleton, Error Alert, dan Empty State yang informatif.
Dokumen Matriks Pemisahan State	10%	Kelengkapan dan ketepatan pemetaan variabel state pada dokumen SRS terhadap pustaka state yang dipilih.

Daftar Pustaka & Referensi Terverifikasi

- Nguyen, T. (2026). Zustand vs Redux: which state manager should you use? Tien Nguyen Engineering Blog.
- CloudRevol. (2026). Top 20 Next.js & React State Management Best Practices Every Developer Should Follow in 2026.
- Egnworks. (2026). Advanced TypeScript Patterns and Zod Schema Validation for Frontend Teams.
- InfoQ. (2026). State of JavaScript 2025: Survey Reveals a Maturing Ecosystem with TypeScript Cementing Dominance.
- SonarSource. (2026). State of Code Developer Survey Report: The Verification Bottleneck in Modern Web Development.
- SVAR UI. (2026). Is React Still the King in 2026? React vs Vue vs Svelte for Modern Web Apps.